

Núcleo de um Coprocessador Gráfico em FPGA para um Console Digital Dedicado na Plataforma DE1-SoC

Bruna de Almeida Nascimento, Carlos Daniel da Silva Jesus, Diego Mercês Almeida

¹ Bacharelado em Engenharia de Computação

UEFS - Universidade Estadual de Feira de Santana, Av. Transnordestina, s/n, Novo Horizonte
Feira de Santana - BA, Brasil - 44036-900

banascimento.uefs@gmail.com, cdanieldsj@gmail.com, diegomerce010@gmail.com

Abstract. *This report describes the design of a graphics coprocessor core for the DE1-SoC platform (Cyclone V), developed in Verilog as Problem #1 of the Digital Systems course. The core implements a tile-based background engine, a 32-sprite engine and a triangle/rectangle rasterizer, combined by a priority compositor and displayed through a VGA controller at 640×480@60Hz over a 320×240 logical resolution. A 32-bit command interface (currently only a demonstration) and dedicated ROM/RAM blocks were designed to allow future control by an ARM/Linux driver. Synthesis results and known limitations – including a timing closure issue and a sprite-transparency bug – are reported.*

Resumo. *Este relatório descreve o projeto do núcleo de um coprocessador gráfico para a plataforma DE1-SoC (Cyclone V), desenvolvido em Verilog como o Problema #1 da disciplina de Sistemas Digitais. O núcleo implementa um motor de background baseado em tiles, um motor de 32 sprites e um rasterizador de triângulos/retângulos, combinados por um compositor de prioridades e exibidos por um controlador VGA a 640×480@60Hz sobre uma resolução lógica de 320×240. Uma interface de comandos de 32 bits (atualmente apenas demonstrativa) e blocos dedicados de ROM/RAM foram projetados para permitir o controle futuro por um driver ARM/Linux. São reportados os resultados de síntese e as limitações conhecidas – incluindo um problema de fechamento de timing e um defeito de transparência dos sprites.*

1. Introdução

Este relatório documenta o desenvolvimento do Problema #1 da disciplina MI - Sistemas Digitais (2026.2), da UEFS, cujo enunciado propõe o desenvolvimento do núcleo de um coprocessador gráfico em FPGA, inspirado na arquitetura de consoles de 16 bits [2]. O núcleo deve suportar um plano de fundo baseado em *tiles*¹, *sprites*² e rasterização de polígonos, sendo preparado para, em etapas futuras, ser controlado por um driver Linux em Assembly ARM³ via acesso por memória mapeada, ou MMIO⁴, e utilizado por uma aplicação de jogo em linguagem C.

¹*Tile*: pequeno bloco gráfico quadrado (neste projeto, de 8×8 pixels) repetido e combinado para formar cenários maiores, técnica clássica de consoles de 16 bits.

²*Sprite*: elemento gráfico independente do plano de fundo (personagem, item, efeito) que pode ser posicionado e movido livremente sobre a cena.

³ARM (*Advanced RISC Machine*): arquitetura de processador utilizada no núcleo de CPU embarcado da placa DE1-SoC (HPS – *Hard Processor System*).

⁴MMIO (*Memory-Mapped I/O*): técnica em que o processador acessa registradores de um periférico (aqui, o coprocessador gráfico) como se fossem posições comuns de memória.

O documento está organizado em dois grandes blocos, conforme as orientações da disciplina [4]: a Seção 2 corresponde ao **Manual do Sistema**, destinado a outro engenheiro que venha a manter ou expandir o projeto, detalhando o problema, os requisitos, a arquitetura, cada bloco funcional, as memórias empregadas, os testes planejados e os resultados de síntese; a Seção 3 corresponde ao **Manual do Usuário**, explicando como instalar, configurar e operar o sistema na placa, com detalhamento completo da pinagem em cada estado da máquina de estados⁵ de demonstração. A Seção 4 traz as conclusões do trabalho, incluindo a lista de limitações e funcionalidades ainda não atendidas.

2. Manual do Sistema

2.1. Declaração do Problema e Requisitos

O sistema final deverá executar um jogo em linguagem C, exibido em monitor VGA⁶, com movimentação do jogador obtida por teclado e mouse conectados à placa, apresentando pontuação, movimentação de objetos e eliminação de agrupamentos [2]. Para viabilizar isso sem que o processador ARM precise desenhar pixel a pixel, o projeto completo é dividido em três componentes: (1) o coprocessador gráfico em FPGA – escopo deste Problema #1 –, (2) um driver em Assembly ARM que futuramente acessará o coprocessador via MMIO, e (3) uma aplicação de jogo em C que futuramente utilizará a API do driver [2].

Os requisitos levantados a partir do enunciado [2] e que orientaram o projeto do núcleo gráfico são resumidos na Tabela 1.

Tabela 1: Síntese dos requisitos do núcleo gráfico.

Categoria	Requisito
Vídeo	Saída VGA 640×480 a ~60 Hz; resolução lógica de 320×240 com duplicação de pixel 2×2.
Arquitetura	Núcleo modular em Verilog, com separação clara entre controle, <i>datapath</i> ⁷ , memórias, motores gráficos e saída de vídeo; reinicialização definida para todos os registradores e memórias; sem instabilidade visual após a inicialização.
Background	Tilemap de 40×30 posições, tiles de 8×8 pixels, no mínimo 256 padrões, com atualização de tile e deslocamento (<i>scroll</i>) horizontal/vertical.
Sprites	Memória de atributos para no mínimo 32 sprites de 16×16 pixels (4 tiles de 8×8); posição X/Y, padrão, habilitação, prioridade, espelhamento H/V e paleta por sprite; prioridade documentada entre sprites sobrepostos.

(continua na próxima página)

⁵Máquina de Estados Finitos (MEF), do inglês *Finite State Machine* (FSM): circuito sequencial que assume um número finito de configurações internas (estados) e transita entre elas de forma síncrona, conforme entradas de controle.

⁶VGA (*Video Graphics Array*): padrão analógico de vídeo, aqui operado em 640×480 pixels a ~60 Hz.

⁷*Datapath* (via de dados): conjunto de registradores, memórias e unidades de processamento por onde os dados fluem, em oposição à lógica de controle que apenas comanda esse fluxo.

(continuação da página anterior)

Categoria	Requisito
Polígonos	Rasterização de triângulos e retângulos preenchidos usando aritmética inteira.
Compositor	Combinação das três camadas por pixel, com no mínimo três níveis de prioridade documentados e transparência aplicada antes da seleção do pixel final.
Paleta	Paleta programável de 256 cores, índice de cor 8 bits, índice 0 reservado como transparente em sprites e polígonos.
Interface	Comandos de 32 bits recebidos pelo núcleo, preparando o mapa de registradores para a futura camada MMIO, sem dependências de uma cena ou jogo específico.

2.2. Arquitetura da Solução Proposta

A arquitetura do núcleo segue o fluxo geral de um coprocessador de vídeo: um gerador de sincronismo produz continuamente as coordenadas de varredura da tela; essas coordenadas alimentam, em paralelo, os três motores gráficos (background, sprites e polígonos), que devolvem um índice de cor de 8 bits cada; um compositor decide, pixel a pixel, qual camada prevalece; e um módulo de paleta converte o índice final em um valor RGB de 24 bits enviado ao conversor digital-analógico (DAC) de vídeo da placa. Paralelamente a esse caminho de dados de vídeo, existe um caminho de controle: uma unidade de controle (FSM de demonstração) seleciona, via chaves da placa, qual motor está sendo operado manualmente para fins de demonstração, e uma interface de comandos de 32 bits – ainda apenas demonstrativa, ver Seção 2.3.4 – representa o formato que futuramente será usado pelo driver ARM.

Esta separação em dois planos – *plano de dados de vídeo* (sempre ativo, gera um pixel por ciclo de clock de vídeo) e *plano de controle/comandos* (ativado por eventos: troca de chave, botão pressionado ou comando de 32 bits recebido) – é o que permite ao núcleo não ter instabilidade visual mesmo quando está sendo configurado.

Diagrama de blocos de alto nível. A Figura 1 apresenta o diagrama de blocos do núcleo gráfico, elaborado pelo grupo. As setas pretas representam as coordenadas de varredura (X, Y), as setas verdes representam sinais de controle distribuídos pela FSM, as setas rosa representam índices de cor/endereço trocados entre os motores e suas memórias. Na seção 2.3.4, falará sobre o módulo da Interface de Comandos, não foi colocado no diagrama, pois é apenas demonstrativo.

(continuação da página anterior)

Sigla	Significado
PAR	Paridade do contador físico de pixel (<code>paridade_x</code>): indica qual dos dois ciclos físicos de 25 MHz corresponde ao pixel lógico atual, usada para compensar a duplicação 2×2 de pixels na saída VGA.
ADR	Endereço enviado a uma memória (RAM ou ROM).
COR	Índice de cor de 8 bits lido de uma memória ou produzido por um motor gráfico.
COR_T	Índice de cor lido da <code>tile_pattern_rom</code> (instanciada internamente pelo motor de background) e repassado como saída do motor.
COR_S	Índice de cor lido da <code>sprite_rom</code> (instanciada internamente pelo motor de sprites) e repassado como saída do motor.
ID	Identificador do tile (índice de padrão gráfico) lido da <code>tilemap_ram</code> .
XD, YD	Coordenadas de depuração (posição do sprite selecionado ou do cursor do rasterizador), mostradas nos displays de 7 segmentos.
MK	Sinal de "marcar ponto" (tecla <code>KEY3</code> no estado do rasterizador de polígonos).
BG, SP, PL	Índices de cor das camadas de B ackground, S prite e P olígono, recebidos pelo compositor.
CF	Cor final, resultado da prioridade aplicada pelo compositor.
RGB	Valor de cor convertido para o formato de saída (8 bits por canal: vermelho, verde e azul).
HS, VS	Sincronismo horizontal (<i>Horizontal Sync</i>) e vertical (<i>Vertical Sync</i>) do sinal VGA.
BL	Sinal de <i>blanking</i> : indica se a varredura está na área visível da tela.
D	Dado de 4 bits (<i>nibble</i>) na entrada do decodificador de display de 7 segmentos.
SEG	Padrão de segmentos gerado na saída do decodificador de 7 segmentos.

2.3. Descrição Detalhada dos Blocos Funcionais

Nesta seção cada módulo Verilog do projeto é descrito individualmente: quais dados recebe, como processa e o que produz na saída.

2.3.1. Geração de Clock e Reset – `clock_reset.v`

Recebe o `CLOCK_50` (50 MHz, oscilador da placa) e o botão físico `KEY[0]` (ativo em nível baixo) como reset. Internamente, divide o clock por 2 usando um flip-flop que inverte de estado a cada borda de subida, produzindo `vga_clk` de 25 MHz – a base de tempo de todo o restante do sistema. O reset é sincronizado ao novo domínio de clock por meio de uma fila de dois flip-flops (`rst_sync1` → `vga_rst_n`), técnica padrão para

evitar metaestabilidade⁸ ao cruzar domínios de clock.

2.3.2. Sincronismo de Vídeo – `controlador_vga.v`

Recebe `vga_clk` e `vga_rst_n`. Mantém dois contadores, um horizontal (0 a 799) e um vertical (0 a 524), implementando a temporização padrão de 640×480@60Hz (área ativa de 640×480, com *front porch*, pulso de sincronismo e *back porch*⁹ definidos como parâmetros). A partir desses contadores gera `o_hsync`, `o_vsync` e `o_blank_n` (1 quando a varredura está na área visível), além das coordenadas físicas `o_x` e `o_y`, sempre grampeadas na faixa válida da tela para não gerar endereços inválidos.

2.3.3. Unidade de Controle / FSM de Demonstração – `fsm_controle.v`

Recebe as chaves `SW[9:8]` (seletor de modo), além dos barramentos brutos `botoes_raw` (`KEY[3:0]`) e `chaves_raw` (`SW[9:0]`). Trata-se de uma máquina de Moore¹⁰ de 3 estados (Background, Sprites, Polígonos) que muda de estado de forma síncrona conforme `seletor`. Sua função principal é *isolar* os barramentos físicos: dependendo do estado atual, a FSM copia apenas o subconjunto relevante de `SW/KEY` para os barramentos de saída dedicados a cada motor (`sw_background`, `key_sprites`, `sw_sprites`, `sw_poligonos`, `key_poligonos`), evitando que uma mesma chave física acione, por engano, dois motores ao mesmo tempo. A pinagem completa de cada estado é detalhada na Seção 3.4.

2.3.4. Interface de Comandos de 32 bits – `decodificador_comandos.v` e `gerador_comandos_demo.v`

Este bloco é apenas uma demonstração do formato de comando futuro, e não uma interface funcional nesta etapa. O `decodificador_comandos` recebe um barramento `comando[31:0]` e um sinal de *strobe*¹¹ válido. Quando válido é 1, o comando é fatiado em `opcode`¹² (`comando[31:28]`) e três parâmetros (`comando[27:20]`, `comando[19:12]` e `comando[11:0]`), definindo cinco operações: `OP_CONFIG_BG` (scroll do background), `OP_SET_TILE` (atualiza um tile do tilemap), `OP_CONFIG_SPR` (escreve atributos de sprite), `OP_RASTER_POLY` (comando de rasterização) e `OP_WRITE_PAL` (escreve uma cor na paleta). Este mapa de opcodes é

⁸Metaestabilidade: estado transitório e imprevisível que um flip-flop pode assumir quando sua entrada muda muito próxima da borda de clock; comum ao cruzar sinais entre domínios de clock diferentes.

⁹*Front porch*, pulso de sincronismo e *back porch*: intervalos de tempo padronizados do sinal VGA entre o fim de uma linha/quadro visível e o início da próxima, necessários para que o monitor reposicione o feixe de varredura.

¹⁰Máquina de Moore: tipo de FSM cujas saídas dependem apenas do estado atual (e não diretamente das entradas), o que garante saídas estáveis durante todo o ciclo de clock.

¹¹*Strobe*: pulso de um único ciclo de clock usado para sinalizar “este dado é válido agora”, em vez de manter um sinal de habilitação constante.

¹²*Opcode (operation code)*: código numérico que identifica qual operação um comando representa, análogo ao opcode de uma instrução de processador.

o "mapa de registradores" citado no enunciado do problema [2], pensado para ser reaproveitado pelo futuro driver ARM. Entretanto, os registradores internos do decodificador (`opcode_atual`, `parametro_a`, `parametro_b`, `parametro_c`) apenas armazenam o valor recebido e **não são lidos por nenhum outro módulo do projeto** – o próprio Quartus confirma isso durante a síntese, emitindo os avisos *Warning (10036): object "opcode_atual" assigned a value but never read* (e equivalentes para os três parâmetros). Ou seja, nesta etapa o decodificador só prova que o formato de 32 bits pode ser recebido e fatiado corretamente; ele ainda não altera o comportamento de nenhum motor gráfico. O `gerador_comandos_demo` simula o processador central apenas para fins de demonstração, enviando ciclicamente, a cada $\sim 0,5$ s, um comando de cada um dos cinco tipos.

2.3.5. Motor de Background – `motor_background.v`

Recebe as coordenadas lógicas `x_logico/y_logico` (0–319, 0–239) e o barramento `controle_sw` (chaves `SW[3:0]`, isoladas pela FSM). Mantém registradores de deslocamento `scroll_h/scroll_v`, atualizados em passos de 8 pixels (um tile inteiro) a $\sim 62,5$ Hz enquanto as chaves de direção estiverem ativas, com *wrap-around*¹³ nas bordas do mapa. A cada pixel, soma o deslocamento à coordenada lógica, calcula a coluna/linha do tile (divisão inteira por 8, feita por deslocamento de bits) e o endereço de 11 bits na `tilemap_ram` (que guarda 1200 posições, 40×30). O identificador do tile lido é concatenado com a posição do pixel dentro do tile (3+3 bits) para formar o endereço de 14 bits da `tile_pattern_rom`, cuja saída é o índice de cor final. Um *pipeline*¹⁴ de dois estágios (`pixel_x_delayed/pixel_y_delayed`) compensa a latência síncrona das duas memórias, garantindo que a cor certa chegue no ciclo de pixel correto.

2.3.6. Motor de Sprites – `motor_sprites.v`

Recebe `x_logico/y_logico`, três botões isolados (`KEY3`, `KEY2`, `KEY1`) e o barramento `controle_sw` de 8 bits (`SW[7:0]`). Internamente, mantém vetores de registradores para os 32 sprites suportados pelo projeto (posição X/Y, padrão gráfico, habilitação, espelhamento H/V, transparência), endereçados pelas 5 chaves menos significativas (`SW[4:0]`). O `KEY1` alterna (com *debounce*¹⁵ de ~ 10 ms) entre dois pares de direção para os botões de movimento `KEY3/KEY2`. Para cada pixel varrido, um laço combinacional percorre os 32 sprites em ordem crescente de índice e seleciona o *primeiro* sprite habilitado que cubra aquele pixel – isso define a prioridade documentada: sprite de índice 0 sempre fica na frente dos demais.

Limitação conhecida – transparência com defeito. A chave `SW7` deveria tornar

¹³*Wrap-around*: quando um contador ou posição ultrapassa seu limite máximo (ou mínimo), ele "dá a volta" e retorna ao início (ou ao fim) da faixa válida, em vez de travar ou gerar um valor inválido.

¹⁴*Pipeline*: técnica de dividir um processamento em estágios encadeados por registradores, de modo que diferentes partes de dados diferentes estejam em estágios diferentes ao mesmo tempo, escondendo a latência de acesso à memória.

¹⁵*Debounce*: filtro digital que ignora oscilações elétricas rápidas e espúrias geradas pelo contato mecânico de um botão, evitando que um único toque seja interpretado como vários.

o índice de cor 0 lido da `sprite_rom` transparente (deixando a camada de baixo aparecer), conforme previsto desde a fase de projeto do motor. Contudo, **esta funcionalidade está com um defeito confirmado pela equipe**: o sprite não fica transparente como deveria. Este é um requisito do enunciado (transparência aplicada antes da composição final) que **não está sendo atendido** nesta versão do núcleo e deve ser corrigido antes da entrega final – possíveis causas a investigar incluem a ordem de avaliação entre `transp_local` e a leitura combinacional de `cor_rom` no bloco de composição do próprio motor de sprites.

Limitação conhecida – sem controle de habilitação por chave. Embora a memória do motor suporte os 32 sprites exigidos pelo enunciado, **não existe, nesta versão, uma chave ou botão dedicado para habilitar ou desabilitar sprites em tempo de execução**. A habilitação (`sprite_en`) é definida apenas uma vez, no reset: os sprites de índice 0 a 3 iniciam habilitados e os de índice 4 a 31 iniciam desabilitados, sem possibilidade de alteração manual durante a demonstração.

2.3.7. Rasterizador de Polígonos – `rasterizador_poligonos.v`

Recebe `x_logico/y_logico`, o barramento `sw[7:0]` e o botão `botao_marcas` (`KEY3`, com sincronização, detecção de borda e *debounce*). Controla um cursor lógico independente (`cursor_x/cursor_y`), movido a ~100 atualizações por segundo pelas chaves `SW[3:0]`, permitindo inclusive movimento diagonal. Cada vez que `KEY3` é pressionado, o cursor atual é capturado como vértice ($X1/Y1$, $X2/Y2$ e, se for triângulo, $X3/Y3$); o tipo de polígono (`SW7`: 0=triângulo, 1=retângulo) é travado no primeiro ponto. Para retângulos, o preenchimento é um teste simples de faixa ($\min_x \leq x \leq \max_x$ e análogo em Y). Para triângulos, o preenchimento usa *edge functions*¹⁶ com aritmética inteira *signed* (com sinal) – $E(A, B, P) = (P_x - A_x)(B_y - A_y) - (P_y - A_y)(B_x - A_x)$ – calculadas para os três lados; o pixel pertence ao triângulo se todas as três funções tiverem o mesmo sinal (todas ≥ 0 ou todas ≤ 0), o que funciona independentemente da orientação (horária ou anti-horária) dos vértices informados. Essa técnica corresponde ao algoritmo de rasterização por funções de borda descrito por Pineda [3], adotado por ser naturalmente paralelizável por pixel e por não exigir divisões, apenas somas e multiplicações inteiras – compatível com a exigência do enunciado de usar aritmética inteira. Um cursor de 1 pixel lógico (cor 255, branca) é sempre desenhado na posição atual para facilitar a mira do usuário.

2.3.8. Compositor de Vídeo – `compositor.v`

Bloco puramente combinacional que recebe os três índices de cor (`cor_bg`, `cor_spr`, `cor_poly`) e aplica a prioridade fixa **polígonos > sprites > background**: se o índice de polígono for diferente de zero, ele prevalece; senão, se o índice de sprite for diferente de zero, ele prevalece; caso contrário, o background é exibido. Como o índice de cor 0

¹⁶*Edge function* (função de borda): expressão linear que, para um dado lado do triângulo, indica de qual semiplano um ponto está; testando os três lados simultaneamente é possível decidir se um pixel está dentro do triângulo sem percorrer suas linhas de varredura uma a uma.

é reservado como transparente para sprites e polígonos, essa cadeia de prioridade implementa ao mesmo tempo a composição das camadas e a regra de transparência exigida no enunciado – exceto pelo defeito de transparência de sprites relatado na Seção 2.3.6.

2.3.9. Índice de Cor / Paleta – `ram_paleta.v`

Decisão de projeto documentada. Em sessão de equipe, decidiu-se não implementar uma paleta programável de 256 entradas como uma memória editável separada. Em vez disso, optou-se por aproveitar diretamente as cores já representáveis pela saída VGA de 8 bits por canal, mapeando o índice de cor de 8 bits diretamente no formato RGB332¹⁷ (3 bits para vermelho, 3 para verde, 2 para azul), já que essas cores “existem” diretamente na própria codificação do índice, sem necessidade de uma tabela de tradução intermediária. O módulo `ram_paleta` apenas replica os bits de cada canal (técnica de *bit replication/fanning*) para preencher os 8 bits por canal exigidos pelo DAC de vídeo, de forma síncrona. Essa decisão simplifica o hardware (sem necessidade de portas de escrita, arbitragem de acesso ou opcode de escrita aplicado) ao custo de abrir mão da reprogramação dinâmica da paleta em tempo de execução – por isso, o opcode `OP_WRITE_PAL` do decodificador de comandos (Seção 2.3.4) permanece apenas ilustrativo nesta versão, já que não há uma tabela para ele escrever. Esse é um ponto de divergência consciente e documentado em relação à redação literal do enunciado (“paleta programável de 256 cores”), listado formalmente entre as funcionalidades não atendidas na Seção 4.

2.3.10. Decodificador de Display de 7 Segmentos – `decodificador_hex.v`

Módulo puramente combinacional e reutilizado seis vezes (`HEX0` a `HEX5`). Converte um nibble de 4 bits em um padrão de 7 segmentos ativo em nível baixo, usado apenas para depuração (mostrar coordenadas do sprite/cursor selecionado e o estado atual da FSM), sem qualquer papel na composição de vídeo.

2.3.11. Integração – `main.v` e `DE1_SOC_golden_top.v`

O módulo `main.v` é o centralizador do núcleo: instancia e conecta todos os blocos acima, expõe as portas físicas da placa (`CLOCK_50`, `KEY`, `SW`, `LEDR`, `HEX0..5`, sinais `VGA_*`) e resolve a lógica de qual par de coordenadas de depuração (sprite ou cursor de polígono) deve ser mostrado nos displays, de acordo com o estado atual da FSM. O `DE1_SOC_golden_top.v` é o top-level padrão fornecido pela Terasic para a DE1-SoC [1], que mapeia *todos* os periféricos físicos da placa (ADC, áudio, HPS, DRAM, GPIO etc.); apenas o subconjunto relevante (clock, chaves, botões, LEDs, displays e VGA) é utilizado pelo núcleo gráfico, sendo os demais periféricos deixados sem uso nesta etapa do projeto.

¹⁷RGB332: formato de cor de 8 bits em que 3 bits representam o vermelho, 3 bits o verde e 2 bits o azul – suficiente para 256 cores distintas sem precisar de uma tabela de tradução separada.

2.4. Memórias ROM e RAM do Projeto

O núcleo utiliza três blocos de memória interna da FPGA (IP¹⁸ `altsyncram` do Quartus), cada um escolhido de acordo com a necessidade de leitura/escrita e o momento em que o conteúdo é definido, conforme a Tabela 3. O índice de cor final (Seção 2.3.9) não usa uma quarta memória, pois é obtido por decodificação combinacional direta, e não por uma tabela armazenada.

Tabela 3: **Memórias internas utilizadas no núcleo gráfico.**

Memória	Tipo	Tamanho	Por quê
<code>tilemap_ram</code>	RAM (leitura/escrita)	1200×8 bits	Guarda o <i>índice do tile</i> em cada uma das 40×30 posições do mapa de fundo. Precisa ser RAM (e não ROM) porque o enunciado exige "permitir alteração do tile associado a cada posição do tilemap" em tempo de execução – futuramente, via comando <code>OP_SET_TILE</code> vindo do driver ARM. Inicializada com <code>tilemap.mif</code> ¹⁹ apenas para ter um cenário padrão visível já no <i>power-up</i> .
<code>tile_pattern_rom</code>	ROM (somente leitura)	16384×8 bits	Guarda os pixels (índices de cor) de até 256 padrões de tile de 8×8 ($256 \times 64 = 16384$). É ROM porque os desenhos dos tiles são recursos gráficos fixos definidos em tempo de projeto (arquivo <code>tilepattern.mif</code>); o que muda em tempo de execução é apenas <i>qual</i> padrão cada posição do tilemap referencia, e isso já é resolvido pela <code>tilemap_ram</code> .

(continua na próxima página)

¹⁸IP core (Intellectual Property core): bloco de hardware pré-projetado e verificado (aqui, gerado pelo assistente do Quartus), reutilizado como uma peça pronta em vez de ser descrito manualmente porta a porta.

¹⁹MIF (Memory Initialization File): arquivo de texto do Quartus que define o conteúdo inicial de uma memória de FPGA no momento da programação, antes de qualquer escrita em tempo de execução.

(continuação da página anterior)

Memória	Tipo	Tamanho	Por quê
<code>sprite_rom</code>	ROM (somente leitura)	8192×8 bits	Guarda os pixels de até 32 padrões de sprite de 16×16 ($32 \times 256 = 8192$), pré-carregados via <code>sprite_patterns.mif</code> . Assim como os tiles, o desenho de cada sprite é um recurso gráfico fixo; os atributos que mudam em tempo real (posição, habilitação, espelhamento) ficam em registradores dentro do próprio <code>motor_sprites</code> , não na ROM.

Em resumo, a regra de projeto adotada foi: **ROM para o conteúdo gráfico fixo** (os desenhos dos tiles e dos sprites, que não mudam durante a execução do jogo) e **RAM para o que precisa ser reconfigurado em tempo real** (o layout do mapa de fundo). Essa mesma lógica de separação motivou a decisão, descrita na Seção 2.3.9, de não instanciar uma quarta memória (RAM) só para a paleta de cores, já que o índice de 8 bits já carrega a cor de forma direta e fixa.

2.5. Organização do Código-Fonte

O projeto é organizado em um arquivo Verilog por bloco funcional (ex.: `motor_background.v`, `motor_sprites.v`, `rasterizador_poligonos.v`), com nomes de sinais em português e comentários de cabeçalho explicando o propósito de cada módulo e de cada porta. As memórias geradas pelo assistente *IP Catalog* do Quartus (`*_rom.v`, `*_ram.v` e seus respectivos `*_bb.v/*_qip`) não devem ser editadas manualmente, sendo tratadas como blocos de IP com uma interface fixa (endereço, clock, dado, `rden/wren`, saída `q`). O projeto Quartus completo (`PBL1-SD_grupo_4.qpf/.qsf`) referencia todos esses arquivos e os `.mif` de inicialização das memórias.

2.6. Testes Planejados

Conforme exigido no enunciado [2], os seguintes cenários funcionais foram planejados para validação por testbench e por demonstração em placa. **Importante: como registrado na Seção 2.7, a equipe não chegou a executar esses testes**; a tabela a seguir descreve a metodologia planejada.

Tabela 4: Cenários de teste planejados.

Cenário	Como seria validado no hardware/estímulo
---------	--

(continua na próxima página)

(continuação da página anterior)

Cenário	Como seria validado no hardware/estímulo
Transparência	No Estado B (Sprites), alternar SW7 com o sprite selecionado sobre o background. Resultado esperado: com SW7=1, o índice de cor 0 do sprite deveria deixar o fundo aparecer. Situação atual: conforme a Seção 2.3.6, essa funcionalidade está com um defeito confirmado – o sprite não fica transparente.
Espelhamento	No Estado B, alternar SW6 (espelhamento horizontal) e SW5 (espelhamento vertical) para um mesmo sprite e observar a inversão do padrão lido da <code>sprite_rom</code> .
Sobreposição	No Estado B, posicionar dois ou mais sprites habilitados na mesma região de tela (movendo-os com KEY3/KEY2) e verificar qual permanece visível.
Prioridade	Combinar as três camadas simultaneamente (fundo com tiles visíveis, um sprite habilitado e um polígono desenhado na mesma região) e confirmar a ordem polígono > sprite > background implementada no <code>compositor</code> .
Troca de buffers	A equipe não implementou um <i>frame buffer</i> ²⁰ ; a imagem é gerada pixel a pixel a cada varredura, sem um buffer de quadro para ser trocado. Este teste não se aplica à versão atual do núcleo.
Comandos inválidos	Como descrito na Seção 2.3.4, o decodificador de comandos ainda é apenas demonstrativo e seus registradores não são lidos por nenhum motor gráfico. Por isso, este teste não pode ser executado nesta versão : mesmo enviando um opcode fora dos cinco definidos, não há efeito observável no vídeo, pois nenhum opcode – válido ou inválido – altera hoje o comportamento dos motores. Este teste só fará sentido após o decodificador passar a escrever nas memórias/regitradores dos motores.

2.7. Resultados de Síntese e Análise

A Tabela 5 resume os resultados de compilação obtidos no Quartus Prime para o dispositivo 5CSEMA5F31C6 (Cyclone V, presente na DE1-SoC), extraídos do relatório do *Fitter*²¹.

²⁰*Frame buffer*: memória que armazena uma imagem completa (um quadro) antes de ela ser exibida, permitindo desenhar a próxima cena enquanto a atual ainda está na tela (*double buffering*).

²¹*Fitter*: etapa do fluxo do Quartus responsável por posicionar (*placement*) e conectar (*routing*) a lógica sintetizada dentro dos recursos físicos disponíveis na FPGA.

Tabela 5. Utilização de recursos (relatório do Fitter).

Recurso	Utilização
Lógica (ALMs ²²)	880 / 32.070 (3%)
Registradores	829
Pinos	241 / 457 (53%)
Bits de memória de bloco	148.864 / 4.065.280 (4%)
Blocos de RAM	19 / 397 (5%)
Blocos DSP ²³	3 / 87 (3%)

O baixo percentual de utilização de lógica e memória mostra ampla margem para expandir o núcleo (por exemplo, adicionar mais camadas de background ou uma paleta RAM real) sem risco de estourar a capacidade do dispositivo.

Limitação conhecida – *timing*²⁴ não fechado. O *Timing Analyzer* do Quartus reporta *setup slack* negativo para o clock `vga_clk` em todos os *corners* de temperatura/tensão analisados (por exemplo, aproximadamente $-10,17$ ns no modelo lento a 85°C), o que indica que, formalmente, o projeto **não fecha *timing*** na frequência atual de 25 MHz, apesar do baixo uso de lógica. **A equipe identificou esse resultado no relatório de compilação, mas não teve conhecimento/tempo suficiente para investigar sua causa raiz ou corrigi-lo até o momento desta entrega.** Como hipótese para trabalho futuro, o caminho combinacional mais longo do projeto provavelmente está no laço de prioridade que varre os 32 sprites em `motor_sprites.v` e/ou no cálculo das *edge functions* do triângulo em `rasterizador_poligonos.v`, ambos calculados inteiramente dentro de um único ciclo de clock. Este ponto é reportado como uma funcionalidade/margem de projeto não atendida, conforme pedido nas orientações do relatório [4].

Ausência de resultados experimentais. A equipe não executou os testes planejados na Seção 2.6 na placa física e não possui registros fotográficos ou em vídeo da demonstração até o momento da elaboração deste relatório. Portanto, as Seções 2.6 e 3.7 descrevem apenas a metodologia de teste e o roteiro de uso planejados, sem resultados experimentais observados. Esta ausência é registrada como uma pendência do projeto, conforme as orientações do relatório [4].

3. Manual do Usuário

3.1. Requisitos de Hardware e Software

- Placa de desenvolvimento Terasic DE1-SoC (FPGA Intel/Altera Cyclone V, modelo 5CSEMA5F31C6) [1];
- Monitor com entrada VGA e cabo VGA compatível;
- Computador com Quartus Prime (usado: versão 20.1.1) instalado, com o cabo USB-Blaster da placa conectado;

²⁴*Timing Analyzer*: ferramenta do Quartus que verifica se todos os caminhos entre registradores conseguem propagar seus sinais dentro de um período de clock, em diferentes combinações (*corners*) de tensão e temperatura de operação. *Setup slack*: margem de tempo, em nanossegundos, entre o instante em que um sinal chega a um registrador e o instante em que ele precisa estar estável antes da próxima borda de clock; um valor negativo significa que o sinal chegaria atrasado demais, violando a garantia de funcionamento correto naquela condição.

- Projeto PBL1-SD_grupo_4.qpf/.qsf e todos os arquivos .v e .mif do repositório do grupo.

3.2. Instalação e Gravação na Placa

1. Abra o arquivo PBL1-SD_grupo_4.qpf no Quartus Prime.
2. Execute *Processing* → *Start Compilation* (ou utilize o arquivo .sof já compilado em output_files/, caso não deseje recompilar).
3. Conecte a DE1-SoC ao computador via USB-Blaster e ligue a placa.
4. Abra o *Programmer* do Quartus, selecione o arquivo PBL1-SD_grupo_4.sof e clique em *Start* para gravar a FPGA.
5. Conecte o monitor VGA à placa antes ou logo após a gravação.
6. Pressione KEY0 para aplicar o reset do sistema e garantir que não há "lixo" de memória visível na tela.



Figura 2. Fotografia da placa DE1-SoC utilizada.

3.3. Seleção de Modo (Estado da FSM) e Reset

O botão KEY0 é fixo como reset geral do sistema (ativo em nível baixo). A seleção de qual motor gráfico está sob controle manual das chaves é feita pelas chaves SW9 e SW8, conforme a Tabela 6. O estado atual também é mostrado, em hexadecimal, no display HEX5, e o valor de SW[9:8] aparece replicado em LEDR[9:8].

Tabela 6. Estados da unidade de controle (FSM de demonstração).

Estado	Código (SW9/SW8)	Motor ativo	Pinos físicos (SW9 / SW8)
A	00	Background	PIN_AE12 / PIN_AD10
B	01	Sprites	PIN_AE12 / PIN_AD10
C	11	Rasterizador de Polígonos	PIN_AE12 / PIN_AD10

3.4. Pinagem por Estado da FSM

Esta seção detalha, para cada estado, exatamente o que cada chave e botão físico faz, incluindo o pino de FPGA correspondente (extraído do arquivo `.qsf` do projeto), para que qualquer pessoa consiga operar a demonstração sem consultar o código-fonte.

3.4.1. Estado A – Motor de Background

Neste estado, as chaves `SW[3:0]` controlam o deslocamento (*scroll*) contínuo do cenário de fundo, em passos de 1 tile (8 pixels) a cada pulso de $\sim 62,5$ Hz enquanto mantidas ativas. A Tabela 7 detalha cada entrada.

Tabela 7. Pinagem do Estado A – Background.

Chave	Valor	Movimento do cenário	Pino FPGA
SW3	1	Rolagem para a esquerda	PIN_AF10
SW2	1	Rolagem para baixo	PIN_AF9
SW1	1	Rolagem para cima	PIN_AC12
SW0	1	Rolagem para a direita	PIN_AB12

Observações: (i) as chaves `SW[9:8]` (seleção de estado) e `SW[7:4]` não têm efeito neste estado; (ii) o `KEY0` continua disponível como reset geral; (iii) as coordenadas de depuração mostradas em `HEX2:HEX0` refletem, neste estado, o próprio valor das chaves de scroll, e não uma posição de sprite ou cursor.

3.4.2. Estado B – Motor de Sprites

Neste estado, os 32 sprites suportados pelo motor (Seção 2.3.6) são editados um de cada vez: as chaves `SW[4:0]` escolhem *qual* sprite (endereço 0 a 31) está sendo editado/movido, enquanto `SW[7:5]` e os botões `KEY[3:1]` controlam seus atributos, conforme a Tabela 8. Não há, nesta versão, uma chave para habilitar/desabilitar sprites (ver limitação na Seção 2.3.6).

Tabela 8: Pinagem do Estado B – Sprites.

Entrada	Função	Pino FPGA
SW0	Bit 0 do endereço do sprite selecionado	PIN_AB12
SW1	Bit 1 do endereço do sprite selecionado	PIN_AC12
SW2	Bit 2 do endereço do sprite selecionado	PIN_AF9
SW3	Bit 3 do endereço do sprite selecionado	PIN_AF10
SW4	Bit 4 do endereço do sprite selecionado (0 a 31 no total)	PIN_AD11
SW5	Espelhamento vertical do sprite selecionado	PIN_AD12
SW6	Espelhamento horizontal do sprite selecionado	PIN_AE11

(continua na próxima página)

(continuação da página anterior)

Entrada	Função	Pino FPGA
SW7	Transparência do sprite selecionado (1 = índice de cor 0 deveria ficar transparente – ver defeito na Seção 2.3.6; 0 = índice 0 aparece como branco de depuração)	PIN_AC9
KEY1	Alterna o par de direções de KEY3/KEY2 (com <i>debounce</i>)	PIN_AA15
KEY2	Movimenta o sprite selecionado (direção depende do modo, ver Tabela 9)	PIN_W15
KEY3	Movimenta o sprite selecionado (direção depende do modo, ver Tabela 9)	PIN_Y16

O par de direções controlado por KEY3/KEY2 depende do modo interno (*modo_direcao*), alternado a cada pressionar de KEY1, conforme a Tabela 9.

Tabela 9. Modos de direção do movimento de sprites.

Modo (após KEY1)	KEY3	KEY2
0 (padrão, após reset)	Move para a esquerda	Move para baixo
1	Move para a direita	Move para cima

Por padrão (após reset), os sprites 0 a 3 estão habilitados e distribuídos em uma grade seguindo o índice do sprite, e os sprites 4 a 31 iniciam desabilitados, permanecendo assim durante toda a demonstração (Seção 2.3.6).

3.4.3. Estado C – Rasterizador de Polígonos

Neste estado, um cursor de 1 pixel percorre a tela e marca os vértices do polígono a ser desenhado, conforme a Tabela 10.

Tabela 10. Pinagem do Estado C – Polígonos.

Entrada	Valor	Ação	Pino FPGA
SW7	0	Próximo polígono marcado será um triângulo	PIN_AC9
SW7	1	Próximo polígono marcado será um retângulo	PIN_AC9
SW3	1	Move o cursor para a esquerda	PIN_AF10
SW2	1	Move o cursor para baixo	PIN_AF9
SW1	1	Move o cursor para cima	PIN_AC12
SW0	1	Move o cursor para a direita	PIN_AB12
KEY3	pressionar	Marca 1 ponto (vértice) na posição atual do cursor	PIN_Y16

O tipo do polígono (SW7) é lido e travado apenas no instante em que o **primeiro** vértice é marcado; alterar SW7 depois disso não afeta o polígono em captura. Retângulos são concluídos com 2 pontos (vértices opostos); triângulos, com 3 pontos. Um novo polígono iniciado durante a captura de outro invalida imediatamente o anterior.

3.5. Indicadores da Placa (LEDs e Displays)

- LEDR[9:8]: replica SW[9:8], ou seja, o estado atual selecionado (00, 01 ou 11).
- LEDR[7:0]: replica SW[7:0], útil para conferir visualmente a leitura das chaves de atributo/controle.
- HEX0, HEX1, HEX2: mostram, em hexadecimal, a coordenada X de depuração – do sprite selecionado no Estado B, do cursor no Estado C, ou das próprias chaves de scroll no Estado A.
- HEX3, HEX4: mostram, em hexadecimal, a coordenada Y de depuração, com o mesmo critério acima (0 fixo no Estado A).
- HEX5: mostra o código do estado atual da FSM (00, 01 ou 11 em hexadecimal, exibidos como 0, 1 ou 3).

3.6. Manual de Uso

O projeto está disponível em um repositório GitHub <https://github.com/Zarconiel/fpga-graphics-coprocessor>, pronto para uso: basta baixar (ou clonar) o repositório, abrir o arquivo PBL1-SD_grupo_4.qpf no Quartus Prime e gravar o núcleo na placa (Seção 3.2). Toda a pinagem das chaves, botões, VGA e displays já vem definida no arquivo .qsf do próprio repositório, conforme detalhado na Seção 3.4, sem necessidade de reatribuição manual de pinos.

Após gravar o núcleo e conectar o monitor VGA, use os passos abaixo para operar cada um dos três modos de demonstração:

1. Pressione KEY0 para resetar o sistema. Com SW9=0, SW8=0 (Estado A), o plano de fundo é exibido; use SW[3:0] para rolar o cenário nas quatro direções, com *wrap-around* nas bordas.
2. Ajuste SW9=0, SW8=1 (Estado B). Com SW[4:0]=00000, mova o sprite 0 com KEY3/KEY2 e alterne o modo de direção com KEY1. A chave SW7 controla a transparência do sprite selecionado (com o defeito conhecido descrito na Seção 2.3.6), e SW6/SW5 controlam o espelhamento.
3. Ainda no Estado B, mova dois dos sprites habilitados por padrão (índices 0 a 3) para a mesma região da tela para visualizar a regra de prioridade (o de menor índice fica por cima).
4. Ajuste SW9=1, SW8=1 (Estado C). Escolha o tipo com SW7, posicione o cursor com SW[3:0] e marque os vértices com KEY3 para desenhar um retângulo e, em seguida, um triângulo.
5. Com um polígono e um sprite habilitados sobre a mesma região do fundo, observe a prioridade final polígono > sprite > background definida no compositor.

4. Conclusão

O núcleo de coprocessador gráfico desenvolvido atende a boa parte dos requisitos funcionais definidos no Problema #1 [2]: gera um sinal de vídeo VGA a partir de uma resolução lógica de 320×240, implementa um motor de background com tilemap editável e scroll, um motor com estrutura para 32 sprites com espelhamento e prioridade bem definida, e um rasterizador de triângulos e retângulos com aritmética inteira baseado em funções de borda [3], todos combinados por um compositor de prioridades fixas. A separação entre

memórias ROM (conteúdo gráfico fixo) e RAM (conteúdo reconfigurável em tempo real) foi uma decisão de projeto central, assim como a decisão, tomada e documentada em sessão de equipe, de usar diretamente o índice de cor no formato RGB332 em vez de uma paleta RAM programável separada.

Conforme exigido nas orientações do relatório [4], seguem listadas as funcionalidades e resultados **não atendidos ou não concluídos** nesta entrega:

- A interface de comandos de 32 bits (`decodificador_comandos`) é apenas demonstrativa: os parâmetros decodificados não são lidos por nenhum motor gráfico, portanto o núcleo ainda não é controlável por comandos externos.
- A transparência de sprites (`SW7`) está com um defeito confirmado: o sprite não fica transparente como deveria.
- Não existe controle (chave/botão) para habilitar ou desabilitar sprites em tempo de execução; a habilitação é fixa desde o reset.
- A paleta de cores não é uma memória programável de 256 entradas, e sim um mapeamento fixo RGB332 – decisão de projeto documentada, mas divergente da redação literal do enunciado.
- O projeto não fecha *timing* em nenhum dos *corners* analisados pelo Quartus (*setup slack* negativo); a causa raiz não foi investigada pela equipe.
- Os testes planejados (Tabela 4) não foram executados na placa física, e não há fotos ou vídeos de demonstração disponíveis para este relatório.
- O cenário de teste "troca de buffers" ainda não foi especificado pela equipe (Seção 2.6).

Como trabalhos futuros, ficam: a correção do defeito de transparência de sprites; a investigação e correção do fechamento de *timing*; a efetiva ligação do decodificador de comandos às memórias/registradores dos motores gráficos; e, em etapas posteriores do projeto geral, a implementação do driver em Assembly ARM e da aplicação de jogo em C que consumirão a interface de comandos aqui projetada.

Referências

- [1] TERASIC. *DE1-SoC User Manual*, rev. F. Terasic Technologies Inc., [S.l.], 2018.
- [2] UNIVERSIDADE ESTADUAL DE FEIRA DE SANTANA. *Problema #1 – 2026.2: Sistema Digital*. Departamento de Tecnologia, Área de Eletrônica, MI - Sistemas Digitais (2026.2).
- [3] PINEDA, J. A Parallel Algorithm for Polygon Rasterization. In: *Proceedings of the 15th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '88)*. ACM, 1988. p. 17–20.
- [4] UNIVERSIDADE ESTADUAL DE FEIRA DE SANTANA. *Orientações para preparação do relatório*. Departamento de Tecnologia, Área de Eletrônica, MI - Sistemas Digitais (2026.2).